

StSATO Security Audit Report

Prepared by	Infinite Trading
Contract	StSATO
Source	contracts/src/stsato.sol
Deployed	Ethereum Mainnet - 0xdeE7f7A032326148E65EC3068F1c9b29E26B75b3
Compiler	Solidity 0.8.26, optimizer_runs=1_000_000, via_ir=true, evm_version=shanghai
OpenZeppelin	v5
Audit Date	May 2026
Test Results	74 / 74 passing

Table of Contents

- [1. Executive Summary](#)
- [2. Lineage and Prior Audit](#)
- [3. Scope](#)
- [4. Protocol Architecture](#)
- [5. Current Contract Characteristics](#)
- [6. Findings](#)
- [7. Security Properties and Invariant Proofs](#)
- [8. Test Validation](#)
- [9. Access Control](#)
- [10. Dependencies](#)
- [11. Deployment Record](#)
- [12. Summary Table](#)

1. Executive Summary

StSATO is a bonding curve liquid staking ERC-20 token backed 1:1 by SATO. The contract is a fork of the **EGGS** implementation from [EGGS Finance](#), adapted for the SATO token on Ethereum mainnet.

The inherited EGGS architecture was audited by **Cantina** in report `report-cantinacode-eggs-0130-2` dated January 30, 2025. That review covered the bonding curve invariant, the collateralised lending system, the liquidation loop, and the price monotonicity constraint.

The StSATO audit and test suite were executed against the forked codepath to reproduce the relevant Cantina coverage on the deployed SATO implementation. All StSATO tests passed.

This report assesses the deployed StSATO contract, its current state, its on-chain invariants, and its observed runtime behavior.

The deployed contract is immutable and ownerless. `renounceOwnership()` is executed atomically inside `setStart()`. There is no upgrade path, no admin key, and no pause mechanism.

2. Lineage and Prior Audit

2.1 EGGS Finance

[EGGS Finance](#) is a DeFi protocol that issues asset-backed tokens with bonding-curve price mechanics. The EGGS token is backed by an underlying asset and supports buying, selling, borrowing at up to 99% LTV, leverage looping, and automated liquidations, which is the exact set of mechanics implemented in StSATO.

StSATO is a direct fork of the EGGS contract, adapted for the SATO ecosystem on Ethereum mainnet. The shared logic consists of the bonding curve pricing model, the `_safetyCheck` price monotonicity invariant, the day-bucket liquidation system, the lending lifecycle, the leverage flow, and the `getMidnightTimestamp` helper.

2.2 Cantina Audit

The original EGGS contract was audited by **Cantina** prior to its mainnet deployment. Report: `report-cantinacode-eggs-0130-2` (January 30, 2025). The Cantina review covered the contract's core invariants, including price monotonicity, collateral solvency, liquidation correctness, and borrow and repay accounting.

3. Scope

File	Description	Status
<code>contracts/src/stsato.sol</code>	Full contract (~550 lines)	Reviewed in full

In scope: all contract logic, including bonding curve math, the lending system, the liquidation loop, fee mechanics, bootstrap, ownership, and all internal and external functions.

Out of scope: the SATO token contract (`0x829f4B62EEBE12Af653b4dD4fFc480966F7d7f09`), frontend integration, and deployment scripts.

4. Protocol Architecture

4.1 Bonding Curve

StSATO is an ERC-20 backed 1:1 by SATO. The price is defined as:

Formula: `price = getBacking() / totalSupply()`

where:

Formula: `getBacking() = SATO.balanceOf(address(this)) + totalBorrowed`

All trade fees accumulate inside the contract as additional SATO backing, causing the price to increase over time. Price is enforced on-chain via `_safetyCheck`, which reverts if `newPrice < lastPrice`.

4.2 Fee Mechanics

Constant	Value	Meaning
<code>buy_fee</code>	990	Buyer receives 99% of the bonding curve output stSATO
<code>sell_fee</code>	990	Seller receives 99% of the bonding curve output SATO

buy_fee_leverage	10	1% mint fee on leverage positions
FEES_BUY / FEES_SELL	125	Trade fee routing used by _sendFees
_sendFees split	1% burn / 99% backing	Of the fee amount, 1% is transferred to 0xdead and 99% remains in the contract

The protocol charges a 1% fee on buys and sells. The collected fee is then split so that 1% is transferred to 0xdead and 99% remains in the contract as additional SATO backing.

4.3 Lending System

Borrowers lock stSATO collateral and receive SATO at up to 99% LTV. Interest is charged upfront. Loans expire on a midnight timestamp bucket; expired loans are liquidated automatically on the next trade via `liquidate()`. Borrowers may always call `repay()` or `closePosition()` regardless of liquidation backlog.

4.4 Key Constants

```
address constant DEAD = 0x00000000000000000000000000000000dEaD;
uint16 public constant sell_fee      = 990;
uint16 public constant buy_fee       = 990;
uint16 public constant buy_fee_leverage = 10;
uint16 private constant FEE_BASE_1000 = 1000;
uint16 private constant FEES_BUY      = 125;
uint16 private constant FEES_SELL     = 125;
```

5. Current Contract Characteristics

Area	Current StSATO Behavior
Backing token	SATO at 0x829f4B62EEBE12Af653b4dD4fFc480966F7d7f09
Token name and symbol	stsato
Network	Ethereum mainnet
Bootstrap recipient	0x00000000000000000000000000000000dEaD
Ownership after launch	Permanently renounced
Fee burn destination	0x00000000000000000000000000000000dEaD
lastPrice after bootstrap	1e18
lastLiquidationDate	Set to the next midnight after deployment
Upgradeability	None
Oracle dependency	None

The contract operates as a self-contained bonding curve market. User-facing buys and sells apply a 1 percent haircut to the curve output. Fee accounting then sends 1 percent of the collected fee bucket to the dead address and retains the remainder as additional backing.

6. Findings

Observation 1. Low severity, unbounded `liquidate()` loop after extended dormancy

Severity	Low
Status	Accepted by design
Location	<code>liquidate()</code>

Description: `liquidate()` iterates day by day from `lastLiquidationDate` until the current block timestamp. The loop is safe under normal operation but its gas cost grows linearly with dormancy.

Impact: This is a liveness constraint, not a loss of funds.

Mitigation present in the contract: `drainLiquidations(maxDays)` provides a permissionless bounded alternative.

Observation 2. Informational, `borrowMore` uses midnight rounding in duration calculation

Severity	Informational
Status	Accepted
Location	<code>borrowMore()</code>

Description: `borrowMore()` derives the remaining term using the midnight helper. Because the helper rounds to the next midnight, the remaining duration can be understated by up to one day.

Impact: Minor user favourable difference in interest fee relative to a strict wall clock calculation.

Observation 3. Informational, `lastLiquidationDate` starts at the next midnight

Severity	Informational
Status	Accepted
Location	Constructor and <code>getMidnightTimestamp()</code>

`lastLiquidationDate` is initialised to the next midnight after deployment. The contract therefore does not process liquidation buckets until that time is reached.

7. Security Properties and Invariant Proofs

7.1 Price Monotonicity

The `_safetyCheck` function is called at the end of every state-changing function and enforces

Invariant: `P_last <= P_new`

where

Formula: `P_new = (B * 1018) / S`

with `B = getBacking()` and `S = totalSupply()`.

The following table shows the effect of each state-changing operation on the backing ratio.

Operation	Backing	Supply	Price direction
buy	Increases	Increases	Strictly increasing
sell	Decreases by less than the supply reduction	Decreases	Strictly increasing
borrow	Neutral plus fee accrual	Unchanged	Increasing
repay	Neutral	Unchanged	Stable
closePosition	Neutral	Unchanged	Stable
extendLoan	Increases via fee	Unchanged	Increasing
leverage	Increases via fee	Increases	Increasing
flashClosePosition	Debt cleared and backing preserved	Decreases	Increasing
liquidate	Debt removed and backing preserved	Decreases	Increasing
removeCollateral	Unchanged	Unchanged	Stable

Proof sketch for `buy` :

Mint output before fees: `m = (X * S) / (B - X)`

After fees:

Mint output after fees: `m_net = (X * S * 990) / ((B - X) * 1000)`

Backing change: `Delta_B = X * (1 - 1/12500)`

Resulting price: `P_new = (B + X * (12499/12500)) / (S + (X * S * 990) / ((B - X) * 1000))`

Since

Inequality: `12499/12500 > 990/1000`

the numerator grows proportionally faster than the denominator. Price therefore increases.

7.2 Collateral Solvency

The `_safetyCheck` function also enforces

Invariant: `balanceOf(address(this)) >= totalCollateral`

This invariant is maintained by construction. `borrow` and `borrowMore` transfer `stSATO` into the contract and increase `totalCollateral` by the same amount. `closePosition` and `removeCollateral` transfer `stSATO` out and decrease `totalCollateral` by the same amount. `liquidate` and `drainLiquidations` burn contract-held `stSATO` and decrease `totalCollateral` by the same amount. Solidity 0.8+ underflow protection prevents arithmetic desync.

7.3 Supply Floor - `totalSupply` Can Never Reach Zero

`setStart()` mints exactly `satoAmount` `stSATO`, with a minimum of `1e18`, permanently to `0xdead`.

Because `0xdead` has no private key and cannot call `burn()`,

Supply floor: $S \geq 10^{18}$

in perpetuity. Division by `totalSupply()` in the bonding curve math is therefore safe.

7.4 Borrower Repayment Always Available

`repay()` and `closePosition()` contain no call to `liquidate()`. They are executable in all conditions, regardless of liquidation backlog, gas costs, or protocol dormancy.

7.5 Overcollateralisation at Liquidation

At borrow time,

Collateral formula: $C = \text{ceil}((X * S) / B)$

Ceiling division ensures

Condition: $C * P \geq X$

Debt is

Debt formula: $D = X * 99/100$

Since price is monotonically non-decreasing,

Condition: $C * P_{\text{current}} \geq X > D$

at all times. Liquidated positions therefore clear with a non-negative effect on the backing ratio.

7.6 No Reentrancy

All public state-changing functions are guarded by `nonReentrant`. State mutations that increment accounting variables precede external `safeTransfer` calls where order allows. The ERC-20 `_burn` call in `sell()` modifies the `msg.sender` balance via internal accounting before any external token movement occurs, and the `nonReentrant` guard prevents any reentrant callback from exploiting an intermediate state.

8. Test Validation

8.1 Test Objective

The test campaign was executed to validate that the deployed `StSATO` fork preserves the critical security properties reviewed in the original `EGGS` audit lineage while also proving that `StSATO` specific deployment state and fee behavior remain correct.

Primary validation goals were:

- 1. Price monotonicity enforcement through `_safetyCheck` .
- 2. Correct accounting across buy, sell, borrow, repay, and liquidation paths.
- 3. Collateral solvency under normal and stressed execution flows.
- 4. Borrower liveness, including guaranteed ability to repay and close positions.
- 5. Correct fee routing and long-term backing accretion behavior.

8.2 Test Environment

Parameter	Value
Framework	Foundry
Solidity version	0.8.26
Optimizer	enabled, 1_000_000 runs
IR pipeline	via_ir=true
EVM target	shanghai
Focused suite	contracts/test/StSAT0.t.sol

8.3 Execution Command

The suite was executed with script files excluded to isolate contract verification tests:

```
forge test --match-path test/StSAT0.t.sol --skip 'script/**'
```

8.4 Result Summary

Metric	Result
Total tests	74
Passed	74
Failed	0
Skipped	0
Critical regressions	None observed

8.5 Coverage by Behavior Class

Behavior class	Representative checks	Outcome
Bootstrap and ownership finalization	setStart backing initialization, dead-address mint, ownership renounce	Pass
Buy and sell math	Mint and redeem amounts, fee effects, monotonic price checks	Pass

Borrow lifecycle	Borrow creation, borrow-more updates, repayment and close flow	Pass
Liquidation engine	Expired loan processing, chunked drain execution, accounting preservation	Pass
Leverage paths	Leverage minting, flash-close debt settlement behavior	Pass
Invariants and stress loops	Repeated trade loops, backing equalities, price non-decrease checks	Pass

8.6 Audit Interpretation of Test Results

A passing test campaign does not replace line-by-line review, but it materially strengthens confidence that:

1. The mathematical and accounting invariants are preserved across all exercised state transitions.
2. Forked StSATO behavior remains aligned with the expected EGGS architecture.
3. The identified low and informational observations are liveness and ergonomics considerations, not solvency or theft vectors.

9. Access Control

StSATO follows an owner-minimized architecture that transitions to ownerless operation at bootstrap.

Control surface	State
Constructor owner	Set initially
Privileged initialization	setStart(uint256) callable by owner before start
Ownership after setStart	Permanently renounced
Upgrade hooks	None
Pausable controls	None

Security implication:

1. No privileged actor can later change fee constants, alter liquidation logic, or seize user funds.
2. Operational risk is shifted from governance risk to immutable-code risk.
3. Any future behavior changes require redeployment rather than admin mutation.

10. Dependencies

The contract uses battle-tested OpenZeppelin v5 modules and internal arithmetic checks from Solidity 0.8.x.

Dependency	Role in security model
ERC20Burnable	Standardized supply accounting and burn semantics
SafeERC20	Hardened token transfer calls to the SATO token
Ownable	Controlled one-time bootstrap then full ownership renounce

ReentrancyGuard	Prevents reentrant entry across external state transitions
Solidity 0.8+ checks	Built-in overflow and underflow protections

No external price oracles are required. No proxy framework is used. No off-chain signer is needed for core solvency logic.

11. Deployment Record

11.1 Deployment Metadata

| **setStart** block | 25152640 || **Bootstrap amount** | 1e18 SATO (1 SATO) || **Etherscan** | <https://etherscan.io/address/0xdee7f7a032326148e65ec3068f1c9b29e26b75b3> |

11.2 Post-Deployment State Verification

The following checks were taken from the deployed mainnet contract state after initialization and source verification:

Check	Result
owner()	0x00 ✓
start()	true ✓
lastPrice()	1000000000000000000 (1e18) ✓
getBacking()	1e18 SATO ✓
totalSupply()	1e18 stSATO (all held by 0xdead) ✓
lastLiquidationDate()	1779494400 (2026-05-23 00:00 UTC) ✓
Source verified on Etherscan	✓

12. Summary Table

#	Title	Severity	Status
6.1	Unbounded liquidate() loop after extreme dormancy	Low	Accepted
6.2	borrowMore duration uses midnight rounding	Informational	Accepted
6.3	lastLiquidationDate starts at the next midnight	Informational	Accepted

No permanent lock of funds is possible. No combination of inputs can permanently prevent a borrower from repaying, a holder from selling, or the protocol from resuming activity.